can be used in the same way as a function defined in a built-in procedural language. Such functions can be used for defining triggers, operators, aggregates, etc. A standard PostgreSQL distribution currently contains four procedural languages, such as PL/pgSQL, PL/Tcl, PL/Perl and PL/Python.

Defining a procedural language involves specifying a language handler through the *CREATE LANGUAGE* command. Upon receiving a request to execute a user-defined function in a procedural language, the PostgreSQL server invokes the language handler for the corresponding procedural language with the source code of the function. It is the responsibility of the language handler to interpret the code in a meaningful way and perform the specified operations by interacting with the core server modules. PostgreSQL exposes certain hooks, which can be exploited by the language handler to function effectively.

Apart from the language handler, a language specification can provide two other functions—validator and inline handler. A validator is required to check syntactic correctness and the argument types of the source text of a function at the time of defining the function. An inline handler is invoked when an anonymous code block written in the corresponding procedural language is encountered. It is required to execute the code of the anonymous block provided. The language implementation can be provided in the form of a shared library, usually written in a low-level language like C.

User-defined procedural languages are helpful if the user wants to convert a lot of application logic written in programming languages into PostgreSQL stored functions or procedures.

## Extensions

A PostgreSQL extension is a way to pack definitions of related objects like functions, types, operators, casts, etc, together. An extension is specified using an SQL script, which defines various objects; a control file, which contains basic information about the extension; and an optional shared library file, which contains the implementation (if required) of objects written in a low-level language like C. With these three files, you can load the objects using a simple *CREATE EXTENSION* command. PostgreSQL remembers all the objects created by the SQL script to be part of the extension, and will drop all those objects when the *DROP EXTENSION* command is issued. The objects in an extension are not dumped individually, but as an extension. While restoring an extension, the objects are re-created using the extension specification files. The standard PostgreSQL distribution provides a lot of useful extensions. PGXN, the PostgreSQL Extension Network, is a central distribution system for PostgreSQL extension libraries. It hosts many PostgreSQL extensions.

## User-defined foreign data wrappers

A foreign data wrapper is a way to make data external to a PostgreSQL server available in PostgreSQL form. A foreign data wrapper is implemented as a function, which acts as a bridge between the foreign data source and the PostgreSQL server. While many foreign data wrappers are available as extensions, users can write their own by specifying a foreign data handler for a given foreign data source.

## Final notes

PostgreSQL allows defining objects of almost any kind supported by the PostgreSQL server (functions, aggregates, types, languages and foreign data wrappers no more remain just metadata, but become objects residing in the database). PostgreSQL's ability to embrace external functionality opens up tremendous possibilities for application development. The standard PostgreSQL distribution is not limiting, and PostgreSQL installations can be enhanced in several ways.

The SQL extensibility of PostgreSQL is greatly facilitated by the design of the PostgreSQL execution engine, which is catalogue-driven. Unlike standard relational database systems, PostgreSQL stores information about data-types, functions, operators, procedural languages, access methods, etc, in catalogues, which are similar to tables containing users' data. Various Data Definition Language (DDL) commands change these catalogues, thus extending or modifying the existing functionality.

Proper access permissions must be administered to prevent misuse of the extensions, which could compromise security or affect performance. Bad constructs and un-optimised queries can lead to slower functions, and thus bad performance.

PostgreSQL is a powerful solution, with a variety of features that are flexible, extensible and empowering for anyone. END

### References

[1] *http://www.postgresql.org/docs/9.2/static/index.html*
[2] *http://www.postgresql.org/docs/9.2/static/xindex.html*

**By: Ashutosh Bapat**

Ashutosh works as a technical architect in EnterpriseDB. He is a member of the core team in the Postgres-XC Development Group. He has spoken at various conferences about Postgres-XC. As an academic, Ashutosh has worked as a visiting faculty member in the College of Engineering, Pune. He holds an M.Tech in Computer Science from the Indian Institute of Technology, Bombay.